Docker & Docker Compose Reference

Command cheat sheet + concepts, built while setting up the local MinIO + PySpark stack for the pyspark-lakehouse-aws project.

1. Core Concepts

Image vs. Container

An **image** is a static, inert blueprint (filesystem + config) sitting on disk or in a registry. A **container** is a running instance of an image — an actual process. Nothing executes until you start a container from an image. This is the same relationship as a Lambda image sitting in ECR until an invocation spins up a running instance of it — except with Compose, several containers run at once and stay up, rather than one container per invocation.

What Docker Compose Adds

A single Docker image (e.g. for AWS Lambda) is a sealed, self-contained unit — your code and dependencies baked in, with no awareness of other containers. Docker Compose instead describes a *system* of multiple containers ("services") that need to run together and talk to each other. Compose handles the orchestration you'd otherwise script by hand:

- Pulls/builds one image per service and starts one container per service
- Creates a private network so containers can reach each other by service name (DNS)
- Orders startup via `depends_on` and healthchecks
- Wires up volumes (persistent storage) and bind mounts (live host–container file sharing)

Volumes vs. Bind Mounts

A container's own filesystem is disposable — delete the container, lose anything written inside it. Two ways around that, both used in this project's `docker-compose.yml`:

- **Named volume** (e.g. `minio-data:/data`) — Docker manages the storage location itself (inside Docker Desktop's internal VM on macOS). Survives container removal; only deleted explicitly (`docker compose down -v` or `docker volume rm`). Not directly browsable in Finder.
- **Bind mount** (e.g. `./src:/opt/app/src`) — maps a real folder on your Mac directly into the container. Visible in Finder/`ls` at all times; edits on the host appear instantly inside the container, no rebuild needed.

Service Networking

Every service in a compose file gets a DNS name equal to its service name, resolvable only from inside that project's Docker network. That's why the Spark config in this project uses `http://minio:9000` as the S3 endpoint instead of `localhost:9000` — "minio" resolves to the MinIO container's internal IP, but only from other containers on the same compose network, not from the host machine.

2. Docker Compose Commands

Command	What it does
`docker compose up -d`	Create the network/volumes and start every service's container, in dependency order, detached (background).
`docker compose up`	Same, but attached — logs stream to your terminal, Ctrl+C stops everything.
`docker compose down`	Stop and remove containers + the project network. Named volumes are kept by default.
`docker compose down -v`	Same as above, but also deletes named volumes — permanently wipes MinIO's stored data. Use with care.
`docker compose stop`	Stop containers without removing them (keeps container state, just paused).
`docker compose start`	Restart previously-stopped containers.
`docker compose ps`	List this project's containers and their status.
`docker compose config`	Validate and print the fully-resolved compose file (env vars substituted). Good sanity check before "up".
`docker compose logs`	Show logs from all services.
`docker compose logs -f minio-init`	Follow logs for one service only (e.g. confirm buckets were created).
`docker compose exec spark bash`	Open an interactive shell inside the already-running spark container.
`docker compose restart spark`	Restart a single service without touching the others.
`docker compose pull`	Re-pull the latest images for each service without starting anything.

3. Plain Docker Commands (useful outside Compose too)

Command	What it does
`docker ps`	List running containers (add -a for stopped ones too).
`docker images`	List images cached locally.
`docker logs <container>`	View a single container's logs.
`docker exec -it <container> sh`	Shell into a running container by name/ID (works without Compose).
`docker volume ls`	List all named volumes on the machine.
`docker volume inspect minio-data`	Show a volume's metadata, including its real path inside Docker Desktop's VM.
`docker run --rm -v minio-data:/data alpine ls -la /data`	Peek inside a named volume's contents using a disposable helper container, without going through MinIO itself.
`docker system df`	Show disk space used by images, containers, and volumes.

Command	What it does
`docker system prune`	Remove unused/dangling images, stopped containers, and networks (does not touch named volumes used by a running project).

4. Accessing Files in MinIO

Object data written into MinIO's buckets (e.g. the bronze zone once the ingestion script runs) lives inside the `minio-data` named volume. Four ways to get at it:

A. Web Console (easiest, visual)

Open `http://localhost:9001` in a browser once the stack is up, log in with the credentials from `.env` (`MINIO_ROOT_USER` / `MINIO_ROOT_PASSWORD`). Browse buckets, folders, and objects, download individual files, view metadata — no CLI needed.

B. mc (MinIO Client) — the CLI equivalent of aws s3

```
# one-time: point mc at this MinIO instance (from your host, if mc is installed locally)
mc alias set local http://localhost:9000 $MINIO_ROOT_USER $MINIO_ROOT_PASSWORD

mc ls local/bronze                      # list top-level of the bronze bucket
mc ls --recursive local/bronze/olist    # list everything under a prefix
mc cat local/bronze/olist/orders/part-00000.csv | head   # peek at a file's contents
mc cp local/bronze/olist/orders/part-00000.csv .         # download a file locally
mc du local/bronze                      # size of the bucket
```

C. From inside the spark container (no extra tools needed)

```
docker compose exec spark bash
# then, inside the container, use Spark/Hadoop's own s3a client via a Python shell,
# or use hadoop fs if the image includes Hadoop CLI tools:
hadoop fs -ls s3a://bronze/olist/
```

D. Python / boto3 (from a script, host or container)

```
import boto3

s3 = boto3.client(
    "s3",
    endpoint_url="http://localhost:9000",  # or http://minio:9000 from inside a container
    aws_access_key_id="minioadmin",
    aws_secret_access_key="minioadmin",
)

resp = s3.list_objects_v2(Bucket="bronze", Prefix="olist/")
for obj in resp.get("Contents", []):
    print(obj["Key"], obj["Size"])
```

Note: from your Mac's host (browser, host-installed mc, or a host Python script), MinIO is reachable at `localhost:9000` because that port is published in docker-compose.yml. From *inside* another container (like spark), it must be `http://minio:9000` — the internal Docker network DNS name, which doesn't resolve from the host.

5. Project-Specific Commands (pyspark-lakehouse-aws)

Command	What it does
`docker compose config`	Validate docker-compose.yml before first run.
`docker compose up -d`	Start MinIO, run the one-shot bucket-creation step, then start the Spark container (kept alive via tail -f /dev/null).
`docker compose logs minio-init`	Confirm the bronze/silver/gold buckets were created successfully.
`docker compose exec spark bash`	Get a shell in the Spark container to run the ingestion script interactively.
`config/spark_session.py (imported by your script)`	Builds a SparkSession pre-configured for s3a against MinIO (endpoint, path-style access, credentials from env).
`./scripts/download_data.sh`	Downloads the Olist dataset from Kaggle into data/raw (run on host, requires kaggle CLI + ~/.kaggle/kaggle.json).
`docker compose down`	Tear down when done for the day — MinIO's data persists in the minio-data volume for next time.

Reference generated for the PySpark Lakehouse (AWS) portfolio project — Stage 1: local Docker Compose setup + raw ingestion.